Department of Computer Science and Engineering

CSA10 – CO2 AT2

Software Engineering

Architecture Design Assignment

CampusBite: Smart Campus Food Ordering System

Detail	Information
Student Name	Dhanush
Register Number	192511180
Course Code	CSA10
Assessment	CSA10 – CO2 AT2
Case Study System	CampusBite: Smart Campus Food Ordering System
Date of Submission	27 October 2026

1. Introduction

Software architecture defines the high-level structure of a system: its components, the relationships between them, and the principles that guide their design and evolution. A good architecture ensures that the quality attributes demanded by stakeholders, such as performance, security and scalability, are achieved. This assignment presents the architecture of CampusBite, the campus food ordering system analysed in CO2 AT1.

1.1 Objectives

- Derive architectural drivers from the requirements in AT1.
- Compare candidate architectural styles and justify the selected one.
- Present the high-level architecture, components, data design and deployment view.
- Show how quality attributes are achieved through architectural tactics.

1.2 System Summary

CampusBite lets students and staff browse menus, pre-order food, pay online and collect at a pickup slot. Canteen managers manage menus, stock and orders, and administrators view reports. The system must handle a heavy peak load during the 40-minute lunch break.

2. Architectural Drivers

Architectural drivers are the requirements that most strongly influence the design. They are taken from the functional and non-functional requirements of AT1.

Driver	Source	Architectural Impact
Peak load of 1,500 concurrent users within minutes	NFR-01, NFR-03	Stateless services, caching, horizontal scaling, asynchronous order processing
High availability during 7 AM to 8 PM	NFR-02, NFR-10	Redundant instances, health checks, automatic failover, backups
Secure online payments and personal data	NFR-04, NFR-09, FR-06	API gateway, role-based access, encryption, PCI-compliant external gateway
No lost or duplicate orders or payments	NFR-06	Transactions, idempotent APIs, message queue, payment reconciliation
Integration with campus login and payment provider	Constraints	Well-defined external interfaces and adapters
Modifiability: new canteens, features and payment modes	NFR-03, NFR-08	Modular services with clear boundaries

Driver	Source	Architectural Impact
Multi-device access (mobile and web)	NFR-07	Common REST API consumed by all clients
Low budget, 4-month schedule, small team	Constraints	Simple, proven technologies; avoid unnecessary complexity

3. Architectural Style Selection

3.1 Candidate Styles

Criteria	Monolithic	Layered Modular (Modular Monolith + Services)	Full Microservices
Development speed	Very fast	Fast	Slow; heavy setup
Scalability	Whole application scales together	Scale stateless services and cache independently	Excellent, per service
Fault isolation	Poor	Good (queue decouples order flow)	Excellent
Operational complexity	Low	Medium	High
Team size needed	Small	Small to medium	Large
Maintainability	Degrades as code grows	Good due to clear modules	Good but needs governance
Suitability for CampusBite	Low (peak load risk)	High	Medium (over-engineered)

3.2 Selected Style and Justification

A layered, service-oriented architecture with an API gateway and event-driven order processing is selected. The application is divided into the presentation, business services, data and infrastructure layers. Business functions are separated into six cohesive services (User and Authentication, Menu, Order, Payment, Notification and Reporting) that can be deployed together at first and split further as the system grows.

- It meets the scalability driver: stateless services run in multiple instances behind a load balancer, and a Redis cache absorbs read-heavy menu traffic.
- It meets the reliability driver: a message queue decouples order placement from kitchen and notification processing, so a slow component does not block users.

- It fits the project constraints: a small team can build and operate it within four months, unlike a full microservices system.
- It keeps change cost low: layers and services have clear interfaces, so new payment modes or canteens can be added without rewriting the system.
- It supports the security driver: all access passes through a single gateway where authentication and rate limiting are enforced.

4. High-Level Architecture

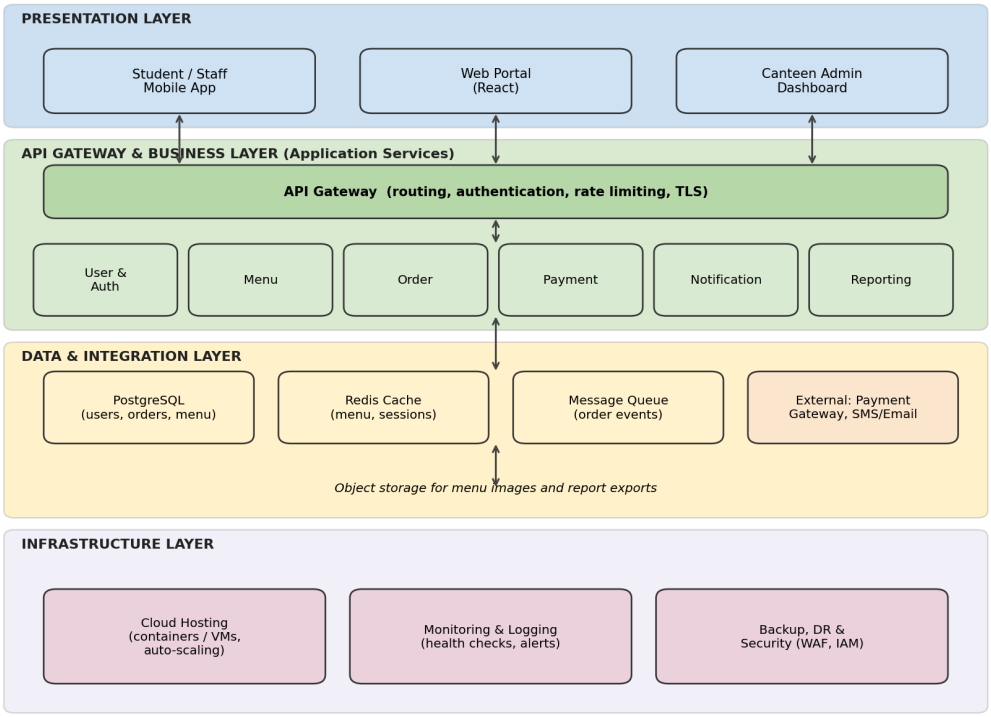


Figure 1: Layered architecture of the CampusBite system

4.1 Layer Description

- **Presentation layer:** Mobile app, React web portal and admin dashboard. They contain no business logic and communicate only through the REST API.
- **API gateway and business layer:** The gateway authenticates every request (JWT), applies rate limits and routes it to the correct service. Services implement the business rules.
- **Data and integration layer:** PostgreSQL stores transactional data, Redis caches menus and sessions, the message queue carries order events, and adapters connect to the payment gateway and SMS / email providers.
- **Infrastructure layer:** Cloud-hosted containers with auto-scaling, monitoring and logging, backup and disaster recovery, and network security.

5. Component Design

Component	Responsibilities	Requirements Served	Interfaces
User and Auth Service	Registration, login, SSO with campus system, role and permission management, token issue	FR-01, FR-15	REST: /auth, /users
Menu Service	Menu, price, availability and stock management; search and filter; cache refresh	FR-02, FR-03, FR-10, FR-11	REST: /menu, /stock
Order Service	Cart, slot validation, order creation, status lifecycle, cancellation, order history	FR-04, FR-05, FR-08, FR-09, FR-13	REST: /orders; publishes OrderCreated events
Payment Service	Payment initiation, callback handling, refunds, reconciliation with the gateway	FR-06, FR-08, NFR-06	REST: /payments; adapter to gateway
Notification Service	Consumes events and sends push, SMS and email messages	FR-07	Queue consumer; adapter to SMS / email
Reporting Service	Aggregates sales and wastage data; exports PDF and Excel	FR-12, FR-16	REST: /reports
API Gateway	Routing, authentication, rate limiting, TLS termination	NFR-04	Single public endpoint

5.1 Order Placement Flow

The order flow shows how the components collaborate and how reliability is achieved:

- The client sends the cart to the API Gateway, which validates the token and forwards the request to the Order Service.
- The Order Service checks stock and slot capacity, then creates the order in 'Pending Payment' state.
- The Payment Service sends the payment request to the external gateway and waits for the callback.
- On success, the order state changes to 'Accepted' within a single database transaction and an OrderCreated event is published to the queue.
- The kitchen dashboard and the Notification Service consume the event, so the user gets confirmation and the kitchen sees the order.

- If the callback is missing, the reconciliation job queries the gateway and either confirms the order or triggers a refund.

6. Data Architecture

6.1 Main Entities

Entity	Key Attributes	Relationships
User	user_id (PK), college_id, name, email, phone, role, password_hash	Places many Orders
Canteen	canteen_id (PK), name, location, open_time, close_time	Has many MenuItems and Slots
MenuItem	item_id (PK), canteen_id (FK), name, category, price, veg_flag, stock_qty, is_active	Belongs to Canteen; appears in OrderItems
Order	order_id (PK), user_id (FK), canteen_id (FK), slot_id (FK), status, total_amount, created_at	Has many OrderItems; has one Payment
OrderItem	order_item_id (PK), order_id (FK), item_id (FK), quantity, unit_price	Links Order and MenuItem
Payment	payment_id (PK), order_id (FK), mode, amount, gateway_ref, status, paid_at	Belongs to Order
PickupSlot	slot_id (PK), canteen_id (FK), start_time, end_time, capacity, booked_count	Belongs to Canteen
Feedback	feedback_id (PK), order_id (FK), item_id (FK), rating, comment	Linked to Order and MenuItem

6.2 Data Management Decisions

- A relational database (PostgreSQL) is used because orders and payments need ACID transactions and strong consistency.
- Read replicas serve reporting queries so that reports do not slow down order processing.
- Menu and availability data are cached in Redis with a short expiry and are invalidated whenever stock changes.
- Sensitive fields such as passwords are hashed, and card details are never stored; only the gateway reference is kept.
- Order and payment records are archived after two years in line with the privacy requirement.

7. Technology Stack

Area	Choice	Reason
Mobile app	Flutter (Android and iOS)	One code base for both platforms
Web frontend	React with responsive design	Reusable components, large community
Backend services	Node.js (Express) or Java Spring Boot	Mature frameworks; team skills; REST support
API gateway	NGINX / Kong	Routing, rate limiting, TLS termination
Database	PostgreSQL	ACID transactions, open source
Cache	Redis	Low-latency reads for menu and sessions
Messaging	RabbitMQ	Reliable asynchronous events and retry
Hosting	Docker containers on a cloud provider with auto-scaling	Elastic capacity for lunch-time peak
CI/CD and monitoring	GitHub Actions, Prometheus, Grafana	Automated testing and deployment; real-time alerts

8. Deployment Architecture

The system is deployed in the cloud in three tiers. A load balancer distributes traffic across at least two instances of every service in different availability zones. Containers are managed by an orchestrator that restarts failed instances and adds instances automatically when CPU usage or request rate crosses a threshold. The database runs with a primary and a standby replica with automatic failover, and backups are taken daily with transaction logs shipped every 15 minutes.

Tier	Elements	Notes
Edge	DNS, CDN, WAF, load balancer	Static content cached; malicious traffic filtered
Application	API gateway and service containers (2 or more instances each)	Stateless, auto-scaled
Data	PostgreSQL primary and replica, Redis, RabbitMQ, object storage	Private subnet; no public access
Operations	Monitoring, centralised logs, alerting, backup storage	On-call alerts to the IT team

9. Quality Attribute Tactics

Quality Attribute	Tactics Used	Requirement
Performance	Redis caching, CDN for images, database indexing, asynchronous processing through the queue, pagination	NFR-01
Scalability	Stateless services, horizontal scaling, load balancer, read replicas	NFR-03
Availability	Redundant instances in two zones, health checks, automatic failover and restart	NFR-02
Security	HTTPS/TLS, JWT authentication, role-based access control, input validation, WAF, hashed passwords, PCI-compliant gateway, audit logs	NFR-04, NFR-09
Reliability	Database transactions, idempotent payment calls, retry with dead-letter queue, reconciliation job	NFR-06
Usability	Responsive UI, consistent design, three-step ordering flow, accessibility guidelines	NFR-05
Modifiability	Layered design, service interfaces, adapters for external systems, configuration instead of code changes	NFR-08
Recoverability	Daily backups, 15-minute log shipping, documented recovery runbook, periodic restore drill	NFR-10
Testability	Dependency injection, mock payment gateway, automated unit, integration and load tests in CI/CD	NFR-08

10. Security Architecture

- Authentication: campus single sign-on or ID and password, with JSON Web Tokens of short validity and refresh tokens.
- Authorisation: role-based access control separating student, staff, kitchen, manager and administrator capabilities.
- Data protection: TLS for data in transit and encryption at rest for databases and backups.
- Payment security: card data is handled only by the external PCI-DSS compliant gateway; callbacks are verified by signature.
- Application security: input validation, parameterised queries to prevent SQL injection, output encoding to prevent XSS, and rate limiting against abuse.
- Monitoring: audit logs of administrative actions and alerts for suspicious activity.

11. Architectural Risks and Trade-offs

Risk / Trade-off	Impact	Mitigation
Payment gateway outage	Users cannot pay	Support multiple modes (UPI, card, wallet); clear error messages; retry
Sudden lunch-time spike beyond forecast	Slow responses	Auto-scaling rules, queue buffering, load testing before launch
Single database is a bottleneck	Throughput limit	Indexing, caching, read replicas; partition by canteen if needed
Eventual consistency between order and notification	Slight delay in messages	Acceptable; order state itself remains strongly consistent
Modular services add operational overhead compared with a monolith	More deployment effort	Containerisation and CI/CD automation
Dependency on campus login service	Login failure if it is down	Fallback local credentials for critical roles

12. Requirement-to-Architecture Mapping

Requirement Group	Realised By
FR-01, FR-15 (users and roles)	User and Auth Service, API Gateway
FR-02, FR-03, FR-10, FR-11 (menu and stock)	Menu Service, Redis cache, PostgreSQL
FR-04, FR-05, FR-08, FR-09, FR-13 (orders)	Order Service, message queue, kitchen dashboard
FR-06 (payments)	Payment Service, external payment gateway adapter
FR-07 (notifications)	Notification Service, SMS / email provider
FR-12, FR-16 (reports and suggestions)	Reporting Service, read replica
NFR-01 to NFR-10	Tactics listed in Section 9 and deployment in Section 8

13. Conclusion

The proposed layered, service-oriented architecture with an API gateway and event-driven order processing satisfies the key drivers identified from the requirement analysis: peak-hour scalability, high availability, secure payments, reliable order handling and easy modification. It is deliberately simple enough for a small team to deliver in four months, yet it leaves room to evolve into finer-grained microservices as the number of canteens and users grows. The architecture is traceable to the requirements of AT1 and provides a clear basis for detailed design, implementation and testing.